

P1609R3

C++ Should Support Just-in-Time Compilation

Published Proposal, 2020-01-13

This version:

<http://wg21.link/p1609>

Issue Tracking:

[Inline In Spec](#)

Author:

[Hal Finkel](#) (Argonne National Laboratory)

Audience:

SG7, SG17

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

C++ has been specifically designed to support the creation of performance-sensitive systems. C++ templates have become a critical piece of this support, allowing the creation of algorithmic specializations to which the application can dispatch based on runtime conditions. However, many applications are pushing this capability to its limit, generating so many specializations using template instantiation that compile times are interfering with programmer productivity. In other cases, programmers simply must settle for suboptimal performance because the ahead-of-time generation of specializations with better performance is not practical given the size of the relevant parameter space. To address these challenges, this paper proposes to add to C++ the capability to support programmer-directed just-in-time (JIT) compilation.

Table of Contents

1 Introduction

2	Changelog
3	Implementation
4	Proposal
4.1	Header <dynamic_instantiation>
4.2	Class diagnostic
4.3	dynamic_function_template_instantiation Operator
4.4	dynamic_template_argument Operator
4.5	for_dynamic_instantiation Attribute
5	Examples
6	On State
7	ABI and Error Handling
8	A Benchmark
8.1	Compile Time
8.2	Runtime Performance
8.3	Overheads
9	Other C++ JIT Approaches
10	Acknowledgments

Issues Index

§ 1. Introduction

C++ is the go-to language for programming performance-sensitive systems. In part, this is due to C++'s design philosophy of "leav[ing] no room for a lower-level language below C++ (except assembler)" (see Appendix A in p0559r0 and Bjarne's 2007 HoPL paper). As C++ programmers, however, we are faced with a fundamental dilemma:

1. A compiler can often generate more-efficient code for algorithms in which some important parameters have (compile-time) known values compared to code for the same algorithms for which those same parameters are known only during program execution. These parameters are

sometimes values (e.g., the number of rows in the matrix is three) and sometimes types. Types here include both types being operated upon and types representing behavioral composition.

2. In some cases, we can template the algorithms on these relevant parameters and instantiate the templates for a large set of the parameters likely to be relevant during program execution. The program can then dispatch to a relevant instantiation during program execution, perhaps falling back to a generic implementation. However, this can have a large compile-time cost. In fact, practical limits on compile time often limit the size of the parameter space which can be covered by instantiated templates.
3. In other cases, the relevant parameter space is so large that a relevant set of instantiations cannot be feasibly selected. In such cases, we're forced to settle for a generic implementation, even if the actual sets of parameters that occur during any particular program execution are not large.

If we had the ability to instantiate templates during program execution, using parameters not known until program execution, we could provide the higher performance of fixed-parameter implementations while simultaneously providing improved compile times. This paper explores the following question: Can we naturally integrate this kind of just-in-time (JIT) compilation capability, including the ability to construct novel template instantiations during program execution, into C++?

Clearly, this kind of JIT compilation capability might not be something that all C++ implementation can provide. In some cases, this is because of technical limitation of the underlying platforms (e.g., in certain kinds of embedded systems). In other cases, this is because the additional risk factors introduced by dynamic compilation are unacceptable in certain environments. Regardless, it seems natural that capabilities in this space would fall into some kind of effective conditionally-supported category.

§ 2. Changelog

- r3: Updated based on feedback from the Belfast meeting.
- r2: Updated based on feedback from the Cologne meeting.
- r1: Updated to discuss overheads in more detail, ABI considerations, and other issues raised on the reflector.

§ 3. Implementation

The proposal below has been implemented, and that implementation is available from the repository: [github:hfinkel/llvm-project-cxxjit](https://github.com/hfinkel/llvm-project-cxxjit). The interface proposed in this version of the paper is available in

branch "cxxjit-ni-9.0". There is some additional documentation on the wiki associated with the repository: [github:hfinkel/llvm-project-cxxjit/wiki](https://github.com/hfinkel/llvm-project-cxxjit/wiki). The data presented below was generated using this implementation.

§ 4. Proposal

After the evening-session discussion in Cologne, and review by EWGI, what follows is an updated proposal for the interface for this functionality. This is not intended to be formal wording, but is intended to be detailed enough to prompt a detailed discussion.

§ 4.1. Header <dynamic_instantiation>

```
#include <source_location> // see 17.8.1
#include <string>           // see 21.3.1

namespace std {
    struct diagnostic;
}
```

ISSUE 1 How should we indicate when we know that the feature won't be available (feature macro, constexpr function, both, something else)?

§ 4.2. Class diagnostic

```
namespace std {

    struct diagnostic {
        const std::string &message() const;
        const std::source_location &location() const;
    };
}
```

The type

`diagnotsic`

meets the Cpp17CopyConstructible, Cpp17CopyAssignable, and Cpp17Destructible requirements (16.5.3.1). Lvalues of type `source_location` are swappable (16.5.3.2).

The class `diagnostic` represents a single compiler diagnostic associated with an evaluation of the `dynamic_function_template_instantiation` operator. These diagnostics might be errors, indicating that the requested instantiation is ill-formed or that the implementation is otherwise unable to create the requested instantiation, or might represent other information the implementation provides about a well-formed instantiation. The content of the message is implementation defined.

§ 4.3. `dynamic_function_template_instantiation` Operator

```
postfix-expression:  
    // add the following...  
    dynamic_function_template_instantiation < id-expression > ( expression-li
```

`dynamic_function_template_instantiation` operator takes an `id-expression`. This `id-expression` must name a non-overloaded function template, and that function template must have a unambiguous function type that is not dependent on any of its template parameters.

[*Note:*

Implementations are encouraged to warn users if the `id-expression` refers to a function-template without the `for_dynamic_instantiation` attribute.

-- end note]

The result of a `dynamic_function_template_instantiation-expression` is an lvalue of an unspecified class type meeting the Cpp17MoveConstructible, Cpp17MoveAssignable, and Cpp17Destructible requirements (16.5.3.1).

The evaluation of the operator shall be an ODR use of the function template identified by

```
id-expression < template-argument-list >
```

where the `template-argument list` is composed from the operator's expression list. The expressions in this list are treated as non-type template parameters except those expressions with the type of some `dynamic_template_argument-operator` evaluation. In the latter case, the template argument is that represented by the expression.

The dynamic instantiation may not succeed. The class type of the result shall be implicitly convertible to bool, and if the dynamic instantiation did not succeed, the value of this conversation shall be false.

The class type shall have a function-call operator of the same type as the named function template, and if the instantiation succeeds, invoking that operator will invoke the dynamically-instantiated function template. If the instantiation does not succeed, and the function-call operator is invoked, the behavior is undefined.

[*Note:*

Invoking the function-call operator is intended to be an operation akin in expense to an indirect function call. Type checking and other related operations should be performed during evaluation of the `dynamic_function_template_instantiation` operator.

-- *end note*]

The class type shall have two const member functions, one named `errors`, and one named `warnings`, and these shall return a reference to an iterable collection of `std::diagnostic` objects.

ISSUE 2 Can the syntax be extended in some way to relax the restriction on overloaded function templates? Maybe allowing an optional parenthesized type list after the id-expression?

ISSUE 3 What should we say about the statefulness of the instantiation process? For example, does the result of friend injection persist across different evaluations of the operator?

ISSUE 4 How might we provide additional information to the implementation to direct the compilation (e.g., compiler-optimization flags or some equivalent)?

ISSUE 5 How can we provide the ability to save/restore the compilation state of particular instantiations (e.g., into / out of a stream)?

Variables that are ODR used, transitively, only by a dynamically-instantiated template, even if such a variable would otherwise have static storage duration, may be destroyed at any point after all relevant result objects of the `dynamic_function_template_instantiation` operators have been destroyed.

[*Note:*

This is intended to allow an implementation to reclaim resources associated with global variables with inline linkage and other related code and data associated with the dynamic instantiations.

-- *end note*]

§ 4.4. `dynamic_template_argument` Operator

```
postfix-expression:  
  // add the following...  
  dynamic_template_argument < template-argument ..._opt >
```

The result of a `dynamic_template_argument`-expression is an lvalue of an unspecified class type meeting the Cpp17CopyConstructible, Cpp17CopyAssignable, and Cpp17Destructible requirements (16.5.3.1). Unlike otherwise, a template argument that is an expression need not be a constant expression. The template argument is evaluated.

[*Note:*

In the aforementioned implementation of this proposal, the operators are named `__clang_dynamic_function_template_instantiation` and `__clang_dynamic_template_argument`.

-- end note]

If the header `<dynamic_instantiation>` is not included prior to any use of the `dynamic_function_template_instantiation` operator or the `dynamic_template_argument` operator, the program is ill-formed.

[*Note:*

This is intended to allow implementations to provide additional definitions in the header that are associated with uses of those operators.

-- end note]

[*Note:*

In the aforementioned implementation of this proposal, this header is not required (an internal header is automatically included with the `-fjit` flag is provided).

-- end note]

ISSUE 6 Should we make the result types of the operators named types provided by the header file? This may be necessary, at least in some cases, to be useful on interfaces.

ISSUE 7 Should `dynamic_template_argument` and `typeid` be unified in some way?

When the template argument is an id-expression naming a template, the result type shall have a member function named `compose` that takes a variable number of arguments. If the argument has the result type of some `dynamic_template_argument` operator, then it is used directly. Otherwise, the `dynamic_template_argument` operator is implicitly applied to each argument. The result of `compose` shall be the same as applying `dynamic_template_argument` to the type formed by combining the id-expression, as a template name, with the template-argument list corresponding to the expressions provided.

[*Note:*

There is no mechanism provided here to detect errors from ill-formed combinations formed using the `compose` member function. This operation is intended to be inexpensive and all error checking deferred until use for some instantiation.

-- *end note*]

§ 4.5. `for_dynamic_instantiation` Attribute

The attribute-token `for_dynamic_instantiation` specifies that a function template is intended for use with the `dynamic_function_template_instantiation` operator. It shall appear at most once in each attribute list and no attribute-argument-clause shall be present. The attribute may be applied to the declarator-id in a function-template declaration.

If a function-template declaration with the `for_dynamic_instantiation` has a function type that is dependent on any of its template parameters, the program is ill-formed. Function-template declarations with the `for_dynamic_instantiation` attribute shall be non-overloadable (12.2).

[*Note:*

In the aforementioned implementation of this proposal, the attribute is named `clang::for_dynamic_instantiation`.

-- *end note*]

First, we need some way to represent templates to enable runtime instantiation without referring to types using strings. We need, essentially, some runtime representation of a dependent template id, where the dependence can now be a runtime dependence.

§ 5. Examples

Thus, we can do the following:

```

template <typename T, int I>
void foo(int a) {
    std::cout << "I was compiled at runtime, I = " << I << "\n";
    std::cout << "I was compiled at runtime, sizeof(T) = " << sizeof(T) << "\n";
}

...

template <int J>
struct A { };

...

auto A_tid = dynamic_template_argument<A>;

...

auto A5 = A_tid.compose(5);

...

int i = ..., j = ...;
auto foo_TI = dynamic_function_template_instantiation<foo>(A5, i);

for (auto &W : foo_TI.warnings()) {
    std::cerr << "warning: " << W.message() << " @ " << W.location().file_name << W.location().line() << ":" << W.location().column() << "\n";
}

if (foo_TI) {
    foo_TI(j);
} else {
    std::cerr << "compilation failed!\n";

    for (auto &W : foo_TI.errors()) {
        std::cerr << "error: " << W.message() << " @ " << W.location().file_name << W.location().line() << ":" << W.location().column() << "\n";
    }
}
}

```

§ 6. On State

Because the instantiation of some templates can affect the instantiation of other templates (e.g., because friend injection can affect later overload resolution), as currently proposed, the implementation of the JIT-compilation functionality cannot be "stateless." While not clearly desirable in general, although it enables some interesting counting tricks (e.g., [Non-Constant Constant-Expressions in C++](#)), this might be particularly undesirable in the JIT-compilation context: It likely makes it more difficult for the JIT-compilation engine to discard its in-memory state even in cases where that might be otherwise advantageous (e.g., to reduce its memory footprint if resource pressure becomes an issue). It is already a non-trivial question as to whether, and under what conditions, the runtime system might be able to determine that a particular instantiation might become unused, thus enabling the freeing of associated resources.

As the JIT-compilation process operates on semantically-processed inputs, and so there is no preprocessor state relevant to the JIT-compilation process. This imposes some restriction on the use of the feature: the JIT-compiled code cannot use differences in the preprocessor state (e.g., whether `__AVX2__` is defined) to customize its behavior (and/or implementation strategy) based on the capabilities of the system on which the program is running. This restriction is likely undesirable, although it is not clear how best to lift the restriction.

§ 7. ABI and Error Handling

It is important that the JIT-compilation facility provide robust error handling capabilities. In this light, although observable within the abstract machine, a production-quality implementation might choose to segregate the JIT-compilation engine into a separate process. JIT compilation can fail because, in addition to compiler bugs, the compilation engine might lack some necessary resources, or the code might otherwise trigger some implementation limit. In addition, compilation might fail because an invalid type string was provided or the provided type or value triggered some semantic error (including triggering a `static_assert`). It might be correct to say that the latter set of conditions are simply undefined behavior, but regardless, providing some ability for the application to handle such errors at the point of instantiation seems useful. No specific mechanism is proposed in this paper, but feedback is certainly desired.

This is related to questions regarding the ABI: If an application is compiled with all of the necessary metadata embedded with in to compile templates used by the proposed operators, does that metadata format and the underlying interface to the JIT-compilation engine that uses it become part of the ABI that the system must support in order to run the application? The answer to this question seems likely to be yes. In practice, this represents another reason why compilation might fail: ABI incompatibility (e.g., running an application on a system with a JIT-compilation engine incompatible with the one

required by the application - one that is too old, for example). At the cost of some (perhaps significant) binary-size overhead, this issue can be avoided by linking the necessary parts of the compiler into the application itself.

Also related to the question of the ABI is that of inter-compiler compatibility: If different translation units that comprise an application's code base are compiled with different compilers, and some of these different translation units use JIT-compilation features, does this necessarily cause problems? Implementation experience is certainly needed here, but it seems likely that an appropriate system ABI could be defined that would allow this kind of combination to work. In the case where the use of different compilers implied the corresponding use of different JIT engines, and we want to ensure properties such as global uniqueness of static local variables in inline functions generated by just-in-time compilation, the system would need provide some way for each JIT engine to register with the others which symbols (functions, global variables, etc.) it had generated. It is unknown what else, if anything, might be required. This seems conceptually similar to the notification functionality that already exists in order to notify a debugger about the newly-generated (JIT-compiled) symbols.

§ 8. A Benchmark

Note: This section uses the interface from a previous revision of the paper...

As a benchmark to illustrate the feature, we'll adapt a benchmark from the [Eigen](https://github.com/eigenteam/eigen-git-mirror/blob/master/bench/benchmark.cpp) library. Specifically, this one: <https://github.com/eigenteam/eigen-git-mirror/blob/master/bench/benchmark.cpp>

We want to look at two aspects: Compile time and runtime performance. Eigen provides a matrix type which can either have compile-time-specific or runtime-specified sizes (i.e., the number of rows and columns).

```
#include <iostream>
#include <string>
#include <chrono>
#include <cstdlib>

#include <Eigen/Core>

using namespace std;
using namespace Eigen;
```

If we wish to support a variant of this benchmark supporting float, double, and long double, and supporting any size at runtime, we can adapt the code as:

```

template <typename T>
void test_aot(int size, int repeat) {
    Matrix<T,Dynamic,Dynamic> I = Matrix<T,Dynamic,Dynamic>::Ones(size, size);
    Matrix<T,Dynamic,Dynamic> m(size, size);
    for(int i = 0; i < size; i++)
        for(int j = 0; j < size; j++) {
            m(i,j) = (i+size*j);
        }

    auto start = chrono::system_clock::now();

    for (int r = 0; r < repeat; ++r) {
        m = Matrix<T,Dynamic,Dynamic>::Ones(size, size) + T(0.00005) * (m + (m^3));
    }

    auto end = chrono::system_clock::now();
    cout << "AoT: " << chrono::duration<double>(end - start).count() << " s\r\n";
}

void test_aot(std::string &type, int size, int repeat) {
    if (type == "float")
        test_aot<float>(size, repeat);
    else if (type == "double")
        test_aot<double>(size, repeat);
    else if (type == "long double")
        test_aot<long double>(size, repeat);
    else
        cout << type << "not supported for AoT\n";
}

```

To do the same thing with the JIT feature, we can write:

```

template <typename T, int size>
[[jit]] void test_jit_sz(int repeat) {
    Matrix<T,size,size> I = Matrix<T,size,size>::Ones();
    Matrix<T,size,size> m;
    for(int i = 0; i < size; i++)
        for(int j = 0; j < size; j++) {
            m(i,j) = (i+size*j);
        }

    auto start = chrono::system_clock::now();

    for (int r = 0; r < repeat; ++r) {
        m = Matrix<T,size,size>::Ones() + T(0.00005) * (m + (m*m));
    }

    auto end = chrono::system_clock::now();
    cout << "JIT: " << chrono::duration<double>(end - start).count() << " s\r\n";
}

void test_jit(std::string &type, int size, int repeat) {
    return test_jit_sz<type, size>(repeat);
}

```

And we can use very-similar code to construct explicit instantiations at compile time, but of course, then we're limited to support for the explicit sizes we have selected.

§ 8.1. Compile Time

Compiling using the implementation linked above on an Intel Xeon E5-2699 using the flags `-march=native -ffast-math -O3`, and measuring compile time using "user" time from the Linux `time` command.

	Time	Time over Base
JIT Only	3.5s	0.92s
(AoT) Single Specialization (double, size = 16)	4.95s	2.37s
(AoT) Single Specialization (double, size = 7)	3.3s	0.72s
(AoT) Single Specialization (double, size = 3)	3.2s	0.62s
(AoT) Single Specialization (double, size = 1)	2.95s	0.37s
(AoT) Two Specializations (double, size = 16) and (double, 7)	5.7s	3.12s

	Time	Time over Base
Generic AoT Only (three floating-point types with dispatch)	9.7s	7.12s
Generic AoT Only (double only)	5.3s	2.72s
Nothing (just the includes and a main function)	2.58s	-

As you can see, the time for generating each specific specialization is essentially additive, and they get more expensive as the fixed matrix sizes get longer. Generating the code for the JIT has a compile-time cost, but it's not even as expensive as a single non-fixed-size implementation.

§ 8.2. Runtime Performance

For (double, size = 3); a repeat count of 40000000. Times as reported by the code (excludes JIT compilation time).

	Time
JIT	1.0s
Single Specialization	1.01s
AoT	8.05s

For (double, size = 7)

	Time
JIT	8.34s
Single Specialization	8.45s
AoT	20s

For (double, size = 16)

	Time
JIT	35.3s
Single Specialization	35.1s
AoT	36.2s

A few trends to notice:

The JIT-generated code is **significantly** faster than the ahead-of-time-generated code for small matrix sizes. The advantage becomes less significant as the matrix sizes become larger. At a high level, this is easy to understand: as the matrix sizes become larger, the value to the optimizer of knowing the exact size decreases.

Thus, using the JIT gives the performance advantages of using many ahead-of-time specializations, and is sometimes even better, with very low compile-time cost.

For more information, see: [arXiv:1904.08555](https://arxiv.org/abs/1904.08555).

§ 8.3. Overheads

There is no overhead to the presence of the JIT-compilation feature in the language when it is not used (although if the feature is enabled, and that causes the compiler to link with the JIT-compilation engine, that can cause both an increase in link time and binary size if the implementation is not aggressive about pruning those dependencies). When the JIT feature is used, however, there are additional costs. Clang's serialized AST can be large, and the current implementation makes no attempt to prune it before embedding it into each object file. For example, in this benchmark, the AoT-only executable is 135 KB in size. The executable making use of the JIT-compilation feature is 12 MB in size, and this does not include the size of the compiled Clang and LLVM libraries. If these libraries are linked to the application statically, instead of dynamically, the size of the binary increases to 75 MB (again, no attempt has yet been made to exclude unnecessary parts of the compiler in the current implementation). In the current implementation, the memory overhead of the JIT-compilation engine is approximately 127 MB, although efforts to prune the serialized AST might be able to reduce that significantly.

Regarding runtime overheads, on an Intel Haswell processor, for the simplest lookup involving a single template argument, the instantiation-cache lookup takes approximately 350 cycles (approximately 140 ns). Should the cache lookup fail, but solely because some types were specified using different names, resolving the instantiation request to the previously-compiled function takes approximately 160 thousand cycles (approximately 65 us). Compiling new instantiations takes, at the very least, tens of millions of cycles (a few milliseconds). The compilation time of a new instantiation, of course, depends heavily on the amount and complexity of the code being compiled (for the example above, this is essentially the time over baseline for each (AoT) single specialization).

§ 9. Other C++ JIT Approaches

Combining C++ with JIT compilation is not new. See:

- Facebook's C++ JIT [talk](#) / [poster](#)
- CERN's [Cling](#)
- From the RuntimeCompiledCPlusPlus project, [A large list](#)

An alternate method for providing an C++ JIT interface is to provide a mechanism, some kind of `std::eval`, which takes a string representing C++ code and evaluates it. Such an interface raises a number of additional questions compared to the interface proposed here, including: Should preprocessor macros be expanded inside of the string, and if so, with what preprocessor state? Do different calls to `std::eval` have any effect on each other (i.e., can one call to `std::eval` define a function called by code in another call to `std::eval`)? If so, are these state changes scoped in any way? If not, would such an interface likely be inefficient to use because it would require reprocessing large amounts of C++ code every time it is invoked? What facilities would need to be provided so that the code designed to construct the to-be-dynamically-executed C++ code could be maintainable?

The flavor of JIT-compilation proposed here aims to provide this capability in keeping with the general C++ philosophy of minimal overhead and explicit programmer control. This makes it a different kind of JIT-compilation than that provided in many of JIT-compiled languages (e.g., Java or Javascript) - in these languages all of the code is JIT-compiled (or interpreted), and sophisticated multi-tiered JIT-compilation, using techniques such as OSR (on-stack replacement) and deoptimization, is used to minimize compilation time spent on all code not measured to be hot (i.e., profiling information is not just used by the JIT-compilation engine itself, but is used to figure out what the JIT-compilation engine should compile in the first place). This proposal focuses on providing a user-directed JIT compilation of code that the programmer already knows is worth compiling during program execution. Future extensions to this proposal might allow for providing some range of possible values/types for template instantiation for use by a JIT-compilation engine capable of performing some online autotuning using that information.

§ 10. Acknowledgments

I'd like to thank David Poliakoff for a lot of testing and feedback on the implementation, and I'd like to thank the many committee members who provided me with feedback on this idea during the Kona meeting and on the reflector (including, but certainly not limited to, Herb Sutter, Chandler Carruth, Olivier Giroux, Vinod Grover, Matthias Kretz, and JF Bastien). I'd also like to thank Michał Dominiak, Nir Friedman, and Connor Waters for providing early feedback online.

This research was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of two U.S. Department of Energy organizations (Office of Science and the National Nuclear Security Administration) responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering, and early testbed platforms, in support of the nation's exascale computing imperative. Additionally, this research used resources of the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.

§ Issues Index

ISSUE 1 How should we indicate when we know that the feature won't be available (feature macro, constexpr function, both, something else)? [↵](#)

ISSUE 2 Can the syntax be extended in some way to relax the restriction on overloaded function templates? Maybe allowing an optional parenthesized type list after the id-expression? [↵](#)

ISSUE 3 What should we say about the statefulness of the instantiation process? For example, does the result of friend injection persist across different evaluations of the operator? [↵](#)

ISSUE 4 How might we provide additional information to the implementation to direct the compilation (e.g., compiler-optimization flags or some equivalent)? [↵](#)

ISSUE 5 How can we provide the ability to save/restore the compilation state of particular instantiations (e.g., into / out of a stream)? [↵](#)

ISSUE 6 Should we make the result types of the operators named types provided by the header file? This may be necessary, at least in some cases, to be useful on interfaces. [↵](#)

ISSUE 7 Should `dynamic_template_argument` and `typeid` be unified in some way? [↵](#)